

Aufgabe 1: Bankkonten

Implementieren Sie die folgenden drei Klassen, welche für eine Banksoftware genutzt werden sollen:

Klasse **Konto**:

- Attribute:
 - Kontoinhaber (String)
 - Kontonummer (String)
 - Kontostand (Kommazahl)
 - 4-stellige Pin (Ganzzahl)
- Methoden:
 - Konstruktor (Initialisierung aller Attribute, Überprüfung des festgelegten Pins auf Plausibilität)
 - Pin ändern (Voraussetzung: Eingabe des bisherigen Pins, anschließend Eingabe eines neuen Pins, Überprüfung des festgelegten Pins auf Plausibilität)

Methoden, die nur nach erfolgreicher Pin-Eingabe ausgeführt werden dürfen:

- Einzahlen (Erhöht den Kontostand um den übergebenen Betrag)
- Auszahlen (Verringert den Kontostand um den übergebenen Betrag)
- Kontoauszug (Gibt den aktuellen Kontostand auf der Konsole aus)

Klasse **Sparkonto**, welche von **Konto** erbt:

- Zusätzliche Attribute:
 - Zinssatz (Kommazahl)
- Zusätzliche Methoden:
 - Jahresabschluss (Erhöht den Kontostand um die Zinsen)
- Zu überschreibende Methoden (werden ebenfalls nur nach erfolgreicher Pin-Eingabe ausgeführt):
 - Konstruktor (Initialisierung aller Attribute)
 - Auszahlen (Auszahlung erfolgt nur, wenn Kontostand nicht ins Minus gerät, ansonsten Warnmeldung)

Klasse **Girokonto**, welche von **Konto** erbt:

- Zusätzliche Attribute:
 - Kreditrahmen (Kommazahl)
 - Sollzinssatz (Kommazahl)
 - Habenzinssatz (Kommazahl)
 - Bearbeitungsgebühr (Kommazahl)
- Zusätzliche Methoden:

- Jahresabschluss (Erhöht bzw. verringert den Kontostand um die Zinsen (Soll-/Habenzinsen)
- Zu überschreibende Methoden (werden ebenfalls nur nach erfolgreicher Pin-Eingabe ausgeführt):
 - Konstruktor (Initialisierung aller Attribute)
 - Auszahlen (Auszahlung erfolgt nur, wenn Kontostand nicht den Kreditrahmen unterschreitet, ansonsten Warnmeldung; Abzug der Bearbeitungsgebühr)
 - Einzahlen (Abzug der Bearbeitungsgebühr)

Aufgabe 2: Bankkonten - Fortsetzung

Erweitern Sie Aufgabe 1. Folgende Änderungen sollen umgesetzt werden:

- Überschreiben Sie in den Klassen `Sparkonto` und `Girokonto` die Methode `Kontoauszug` so, dass diese auch noch die Informationen der zusätzlichen Attribute ausgibt, z. B. den Zinssatz.
- Führen Sie in der Klasse `Konto` ebenfalls die Methode `Jahresabschluss` ein. Diese Methode macht nichts und bleibt daher leer.
- Passen Sie alle Klassen mit `virtual` und `override` so an, dass die dynamische Bindung für alle Methoden funktioniert.

Aufgabe 3: Kontoverwaltung

Implementieren Sie eine Klasse `Kontoverwaltung`. Diese soll die Klassen aus Aufgabe 1 verwenden. Die Klasse enthält als Attribut ein Array aus Adressen (Pointern) des Typs `Konto` und eine Methode, um Konten dem Array hinzuzufügen. Zusätzlich gibt es noch die Methoden `Alle_Kontoauszuege_drucken` und `Jahresabschluss_durchfuehren`. Da es sich hierbei um eine interne Verwaltungssoftware der Bank handelt, wird die im Normalfall notwendige Pin-Eingabe übersprungen.

Testen Sie die Klasse `Kontoverwaltung` in Ihrer `main`-Methode, indem Sie je ein Konto der verschiedenen Kontentypen anlegen und der Kontoverwaltung hinzufügen. Führen Sie einen Jahresabschluss durch und drucken Sie alle Kontoauszüge aus. Prüfen Sie, ob die dynamische Bindung richtig funktioniert. Dies ist der Fall, wenn der Jahresabschluss für jedes Konto entsprechend dem korrekten Kontentyp durchgeführt wird und auch der richtige Kontoauszug gedruckt wird.